Announcement: pandas 3.0 released! Read more in the blog post ×

pandas Getting started User Guide API reference Development Release notes Search ⌘ + K 3.0 (stable)

10 minutes to pandas
Intro to data structures
Essential basic functionality
IO tools (text, CSV, HDF5, …)
PyArrow Functionality
Indexing and selecting data
MultiIndex / advanced indexing
Copy-on-Write (CoW)
Merge, join, concatenate and compare
Reshaping and pivot tables
Working with text data
Working with missing data
Duplicate Labels
Categorical data
Nullable integer data type
Nullable Boolean data type
Chart visualization
Table Visualization
User-Defined Functions (UDFs)
Group by: split-apply-combine
Windowing operations
Time series / date functionality
Time deltas

User Guide > Categorical data

On this page
Object creation
CategoricalDtype
Description
Working with categories
Sorting and order
Comparisons
Operations
Data munging
Getting data in/out
Missing data
Differences to R's `factor`
Gotchas

Categorical data

This is an introduction to pandas categorical data type, including a short comparison with R's `factor`.

`Categoricals` are a pandas data type corresponding to categorical variables in statistics. A categorical variable takes on a limited, and usually fixed, number of possible values (`categories`; `levels` in R). Examples are gender, social class, blood type, country affiliation, observation time or rating via Likert scales.

In contrast to statistical categorical variables, categorical data might have an order (e.g. 'strongly agree' vs. 'agree' or 'first observation' vs. 'second observation'), but numerical operations (additions, divisions, …) are not possible.

All values of categorical data are either in `categories` or `np.nan`. Order is defined by the order of `categories`, not lexical order of the values. Internally, the data structure consists of a `categories` array and an integer array of `codes` which point to the real value in the `categories` array.

The categorical data type is useful in the following cases:

- A string variable consisting of only a few different values. Converting such a string variable to a categorical variable will save some memory, see here.
- The lexical order of a variable is not the same as the logical order ("one", "two", "three"). By converting to a categorical and specifying an order on the categories, sorting and min/max will use the logical order instead of the lexical order, see here.
- As a signal to other Python libraries that this column should be treated as a categorical variable (e.g. to use suitable statistical methods or plot types).

See also the API docs on categoricals.

Object creation

Series creation

Categorical `Series` or columns in a `DataFrame` can be created in several ways:

By specifying `dtype="category"` when constructing a `Series`:

```
In [1]: s = pd.Series(["a", "b", "c", "a"], dtype="category")

In [2]: s
Out[2]:
0    a
1    b
2    c
3    a
dtype: category
Categories (3, str): ['a', 'b', 'c']
```

By converting an existing `Series` or column to a `category` dtype:

```
In [3]: df = pd.DataFrame({"A": ["a", "b", "c", "a"]})

In [4]: df["B"] = df["A"].astype("category")

In [5]: df
Out[5]:
   A  B
0  a  a
1  b  b
2  c  c
3  a  a
```

By using special functions, such as `cut()`, which groups data into discrete bins. See the example on tiling in the docs.

```
In [6]: df = pd.DataFrame({"value": np.random.randint(0, 100, 20)})

In [7]: labels = ["{0} - {1}".format(i, i + 9) for i in range(0, 100, 10)]

In [8]: df["group"] = pd.cut(df.value, range(0, 105, 10), right=False, labels=label

In [9]: df.head(10)
Out[9]:
   value    group
0     65  60 - 69
1     49  40 - 49
2     56  50 - 59
3     43  40 - 49
4     43  40 - 49
5     91  90 - 99
6     32  30 - 39
7     87  80 - 89
8     36  30 - 39
9      8    0 - 9
```

By passing a `pandas.Categorical` object to a `Series` or assigning it to a `DataFrame`.

```
In [10]: raw_cat = pd.Categorical(
   ....:     [None, "b", "c", None], categories=["b", "c", "d"], ordered=False
   ....: )
   ....: 

In [11]: s = pd.Series(raw_cat)

In [12]: s
Out[12]:
0    NaN
1      b
2      c
3    NaN
dtype: category
Categories (3, str): ['b', 'c', 'd']

In [13]: df = pd.DataFrame({"A": ["a", "b", "c", "a"]})

In [14]: df["B"] = raw_cat

In [15]: df
Out[15]:
   A    B
0  a  NaN
1  b    b
2  c    c
3  a  NaN
```

Categorical data has a specific `category` dtype:

```
In [16]: df.dtypes
Out[16]:
A         str
B    category
dtype: object
```

DataFrame creation

Similar to the previous section where a single column was converted to categorical, all columns in a `DataFrame` can be batch converted to categorical either during or after construction.

This can be done during construction by specifying `dtype="category"` in the `DataFrame` constructor:

```
In [17]: df = pd.DataFrame({"A": list("abca"), "B": list("bccd")}, dtype="category"

In [18]: df.dtypes
Out[18]:
A    category
B    category
dtype: object
```

Note that the categories present in each column differ; the conversion is done column by column, so only labels present in a given column are categories:

```
In [19]: df["A"]
Out[19]:
0    a
1    b
2    c
3    a
Name: A, dtype: category
Categories (3, str): ['a', 'b', 'c']

In [20]: df["B"]
Out[20]:
0    b
1    c
2    c
3    d
Name: B, dtype: category
Categories (3, str): ['b', 'c', 'd']
```

Analogously, all columns in an existing `DataFrame` can be batch converted using `DataFrame.astype()`:

```
In [21]: df = pd.DataFrame({"A": list("abca"), "B": list("bccd")})

In [22]: df_cat = df.astype("category")

In [23]: df_cat.dtypes
Out[23]:
A    category
B    category
dtype: object
```

This conversion is likewise done column by column:

```
In [24]: df_cat["A"]
Out[24]:
0    a
1    b
2    c
3    a
Name: A, dtype: category
Categories (3, str): ['a', 'b', 'c']

In [25]: df_cat["B"]
Out[25]:
0    b
1    c
2    c
3    d
Name: B, dtype: category
Categories (3, str): ['b', 'c', 'd']
```

Controlling behavior

In the examples above where we passed `dtype='category'`, we used the default behavior:

1. Categories are inferred from the data.
2. Categories are unordered.

To control those behaviors, instead of passing `'category'`, use an instance of `CategoricalDtype`.

```
In [26]: from pandas.api.types import CategoricalDtype

In [27]: s = pd.Series([None, "b", "c", None])

In [28]: cat_type = CategoricalDtype(categories=["b", "c", "d"], ordered=True)

In [29]: s_cat = s.astype(cat_type)

In [30]: s_cat
Out[30]:
0    NaN
1      b
2      c
3    NaN
dtype: category
Categories (3, str): ['b' < 'c' < 'd']
```

Similarly, a `CategoricalDtype` can be used with a `DataFrame` to ensure that categories are consistent among all columns.

```
In [31]: from pandas.api.types import CategoricalDtype

In [32]: df = pd.DataFrame({"A": list("abca"), "B": list("bccd")})

In [33]: cat_type = CategoricalDtype(categories=list("abcd"), ordered=True)

In [34]: df_cat = df.astype(cat_type)

In [35]: df_cat["A"]
Out[35]:
0    a
1    b
2    c
3    a
Name: A, dtype: category
Categories (4, str): ['a' < 'b' < 'c' < 'd']

In [36]: df_cat["B"]
Out[36]:
0    b
1    c
2    c
3    d
Name: B, dtype: category
Categories (4, str): ['a' < 'b' < 'c' < 'd']
```

> **Note**
>
> To perform table-wise conversion, where all labels in the entire `DataFrame` are used as categories for each column, the `categories` parameter can be determined programmatically by `categories = pd.unique(df.to_numpy().ravel())`.

If you already have `codes` and `categories`, you can use the `from_codes()` constructor to save the factorize step during normal constructor mode:

```
In [37]: splitter = np.random.choice([0, 1], 5, p=[0.5, 0.5])

In [38]: s = pd.Series(pd.Categorical.from_codes(splitter, categories=["train", "te
```

Regaining original data

To get back to the original `Series` or NumPy array, use `Series.astype(original_dtype)` or `np.asarray(categorical)`:

```
In [39]: s = pd.Series(["a", "b", "c", "a"])

In [40]: s
Out[40]:
0    a
1    b
2    c
3    a
dtype: str

In [41]: s2 = s.astype("category")

In [42]: s2
Out[42]:
0    a
1    b
2    c
3    a
dtype: category
Categories (3, str): ['a', 'b', 'c']

In [43]: s2.astype(str)
Out[43]:
0    a
1    b
2    c
3    a
dtype: str

In [44]: np.asarray(s2)
Out[44]: array(['a', 'b', 'c', 'a'], dtype=object)
```

> **Note**
>
> In contrast to R's `factor` function, categorical data is not converting input values to strings; categories will end up the same data type as the original values.

> **Note**
>
> In contrast to R's `factor` function, there is currently no way to assign/change labels at creation time. Use `categories` to change the categories after creation time.

CategoricalDtype

A categorical's type is fully described by

1. `categories`: a sequence of unique values and no missing values
2. `ordered`: a boolean

This information can be stored in a `CategoricalDtype`. The `categories` argument is optional, which implies that the actual categories should be inferred from whatever is present in the data when the `pandas.Categorical` is created. The categories are assumed to be unordered by default.

```
In [45]: from pandas.api.types import CategoricalDtype

In [46]: CategoricalDtype(["a", "b", "c"])
Out[46]: CategoricalDtype(categories=['a', 'b', 'c'], ordered=False, categories_dty

In [47]: CategoricalDtype(["a", "b", "c"], ordered=True)
Out[47]: CategoricalDtype(categories=['a', 'b', 'c'], ordered=True, categories_dtype

In [48]: CategoricalDtype()
Out[48]: CategoricalDtype(categories=None, ordered=False, categories_dtype=None)
```

A `CategoricalDtype` can be used in any place pandas expects a `dtype`. For example `pandas.read_csv()`, `pandas.DataFrame.astype()`, or in the `Series` constructor.

> **Note**
>
> As a convenience, you can use the string `'category'` in place of a `CategoricalDtype` when you want the default behavior of the categories being unordered, and equal to the set values present in the array. In other words, `dtype='category'` is equivalent to `dtype=CategoricalDtype()`.

Equality semantics

Two instances of `CategoricalDtype` compare equal whenever they have the same categories and order. When comparing two unordered categoricals, the order of the `categories` is not considered. Note that categories with different dtypes are not the same.

```
In [49]: c1 = CategoricalDtype(["a", "b", "c"], ordered=False)

# Equal, since order is not considered when ordered=False
In [50]: c1 == CategoricalDtype(["b", "c", "a"], ordered=False)
Out[50]: True

# Unequal, since the second CategoricalDtype is ordered
In [51]: c1 == CategoricalDtype(["a", "b", "c"], ordered=True)
Out[51]: False
```

All instances of `CategoricalDtype` compare equal to the string `'category'`.

```
In [52]: c1 == "category"
Out[52]: True
```

Notice that the `categories_dtype` should be considered, especially when comparing with two empty `CategoricalDtype` instances.

```
In [53]: c2 = pd.Categorical(np.array([], dtype=object))

In [54]: c3 = pd.Categorical(np.array([], dtype=float))

In [55]: c2.dtype == c3.dtype
Out[55]: False
```

Description

Using `describe()` on categorical data will produce similar output to a `Series` or `DataFrame` of type `string`.

```
In [56]: cat = pd.Categorical(["a", "c", "c", np.nan], categories=["b", "a", "c"])

In [57]: df = pd.DataFrame({"cat": cat, "s": ["a", "c", "c", np.nan]})

In [58]: df.describe()
Out[58]:
       cat  s
count    3  3
unique   2  2
top      c  c
freq     2  2

In [59]: df["cat"].describe()
Out[59]:
count     3
unique    2
top       c
freq      2
Name: cat, dtype: object
```

Working with categories

Categorical data has a `categories` and a `ordered` property, which list their possible values and whether the ordering matters or not. These properties are exposed as `s.cat.categories` and `s.cat.ordered`. If you don't manually specify categories and ordering, they are inferred from the passed arguments.

```
In [60]: s = pd.Series(["a", "b", "c", "a"], dtype="category")

In [61]: s.cat.categories
Out[61]: Index(['a', 'b', 'c'], dtype='str')

In [62]: s.cat.ordered
Out[62]: False
```

It's also possible to pass in the categories in a specific order:

```
In [63]: s = pd.Series(pd.Categorical(["a", "b", "c", "a"], categories=["c", "b", "

In [64]: s.cat.categories
Out[64]: Index(['c', 'b', 'a'], dtype='str')

In [65]: s.cat.ordered
Out[65]: False
```

> **Note**
>
> New categorical data are **not** automatically ordered. You must explicitly pass `ordered=True` to indicate an ordered `Categorical`.

> **Note**
>
> The result of `unique()` is not always the same as `Series.cat.categories`, because `Series.unique()` has a couple of guarantees, namely that it returns categories in the order of appearance, and it only includes values that are actually present.
>
> ```
> In [66]: s = pd.Series(list("babc")).astype(CategoricalDtype(list("abcd")))
>
> In [67]: s
> Out[67]:
> 0    b
> 1    a
> 2    b
> 3    c
> dtype: category
> Categories (4, str): ['a', 'b', 'c', 'd']
>
> # categories
> In [68]: s.cat.categories
> Out[68]: Index(['a', 'b', 'c', 'd'], dtype='str')
>
> # uniques
> In [69]: s.unique()
> Out[69]:
> ['b', 'a', 'c']
> Categories (4, str): ['a', 'b', 'c', 'd']
> ```

Renaming categories

Renaming categories is done by using the `rename_categories()` method:

```
In [70]: s = pd.Series(["a", "b", "c", "a"], dtype="category")

In [71]: s
Out[71]:
0    a
1    b
2    c
3    a
dtype: category
Categories (3, str): ['a', 'b', 'c']

In [72]: new_categories = ["Group %s" % g for g in s.cat.categories]

In [73]: s = s.cat.rename_categories(new_categories)

In [74]: s
Out[74]:
0    Group a
1    Group b
2    Group c
3    Group a
dtype: category
Categories (3, str): ['Group a', 'Group b', 'Group c']

# You can also pass a dict-like object to map the renaming
In [75]: s = s.cat.rename_categories({1: "x", 2: "y", 3: "z"})

In [76]: s
Out[76]:
0    Group a
1    Group b
2    Group c
3    Group a
dtype: category
Categories (3, str): ['Group a', 'Group b', 'Group c']
```

> **Note**
>
> In contrast to R's `factor`, categorical data can have categories of other types than string.

Categories must be unique or a `ValueError` is raised:

```
In [77]: try:
   ....:     s = s.cat.rename_categories([1, 1, 1])
   ....: except ValueError as e:
   ....:     print("ValueError:", str(e))
   ....: 
ValueError: Categorical categories must be unique
```

Categories must also not be `NaN` or a `ValueError` is raised:

```
In [78]: try:
   ....:     s = s.cat.rename_categories([1, 2, np.nan])
   ....: except ValueError as e:
   ....:     print("ValueError:", str(e))
   ....: 
ValueError: Categorical categories cannot be null
```

Appending new categories

Appending categories can be done by using the `add_categories()` method:

```
In [79]: s = s.cat.add_categories([4])

In [80]: s.cat.categories
Out[80]: Index(['Group a', 'Group b', 'Group c', 4], dtype='object')

In [81]: s
Out[81]:
0    Group a
1    Group b
2    Group c
3    Group a
dtype: category
Categories (4, object): ['Group a', 'Group b', 'Group c', 4]
```

Removing categories

Removing categories can be done by using the `remove_categories()` method. Values which are removed are replaced by `np.nan`.:

```
In [82]: s = s.cat.remove_categories([4])

In [83]: s
Out[83]:
0    Group a
1    Group b
2    Group c
3    Group a
dtype: category
Categories (3, object): ['Group a', 'Group b', 'Group c']
```

Removing unused categories

Removing unused categories can also be done:

```
In [84]: s = pd.Series(pd.Categorical(["a", "b", "a"], categories=["a", "b", "c", "

In [85]: s
Out[85]:
0    a
1    b
2    a
dtype: category
Categories (4, str): ['a', 'b', 'c', 'd']

In [86]: s.cat.remove_unused_categories()
Out[86]:
0    a
1    b
2    a
dtype: category
Categories (2, str): ['a', 'b']
```

Setting categories

If you want to do remove and add new categories in one step (which has some speed advantage), or simply set the categories to a predefined scale, use `set_categories()`.

```
In [87]: s = pd.Series(["one", "two", "four", "-"], dtype="category")

In [88]: s
Out[88]:
0     one
1     two
2    four
3       -
dtype: category
Categories (4, str): ['-', 'four', 'one', 'two']

In [89]: s = s.cat.set_categories(["one", "two", "three", "four"])

In [90]: s
Out[90]:
0     one
1     two
2    four
3     NaN
dtype: category
Categories (4, str): ['one', 'two', 'three', 'four']
```

> **Note**
>
> Be aware that `Categorical.set_categories()` cannot know whether some category is omitted intentionally or because it is misspelled or (under Python3) due to a type difference (e.g., NumPy S1 dtype and Python strings). This can result in surprising behaviour!

Sorting and order

If categorical data is ordered (`s.cat.ordered == True`), then the order of the categories has a meaning and certain operations are possible. If the categorical is unordered, `.min()/.max()` will raise a `TypeError`.

```
In [91]: s = pd.Series(pd.Categorical(["a", "b", "c", "a"], ordered=False))

In [92]: s = s.sort_values()

In [93]: s = pd.Series(["a", "b", "c", "a"]).astype(CategoricalDtype(ordered=True))

In [94]: s = s.sort_values()

In [95]: s
Out[95]:
0    a
3    a
1    b
2    c
dtype: category
Categories (3, str): ['a' < 'b' < 'c']

In [96]: s.min(), s.max()
Out[96]: ('a', 'c')
```

You can set categorical data to be ordered by using `as_ordered()` or unordered by using `as_unordered()`. These will by default return a *new* object.

```
In [97]: s.cat.as_ordered()
Out[97]:
0    a
3    a
1    b
2    c
dtype: category
Categories (3, str): ['a' < 'b' < 'c']

In [98]: s.cat.as_unordered()
Out[98]:
0    a
3    a
1    b
2    c
dtype: category
Categories (3, str): ['a', 'b', 'c']
```

Sorting will use the order defined by categories, not any lexical order present on the data type. This is even true for strings and numeric data:

is even true for strings and numeric data:

```
In [99]: s = pd.Series([1, 2, 3, 1], dtype="category")

In [100]: s = s.cat.set_categories([2, 3, 1], ordered=True)

In [101]: s
Out[101]: 
0    1
1    2
2    3
3    1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [102]: s = s.sort_values()

In [103]: s
Out[103]: 
1    2
2    3
0    1
3    1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [104]: s.min(), s.max()
Out[104]: (np.int64(2), np.int64(1))
```

Reordering

Reordering the categories is possible via the `Categorical.reorder_categories()` and the `Categorical.set_categories()` methods. For `Categorical.reorder_categories()`, all old categories must be included in the new categories and no new categories are allowed. This will necessarily make the sort order the same as the categories order.

```
In [105]: s = pd.Series([1, 2, 3, 1], dtype="category")

In [106]: s = s.cat.reorder_categories([2, 3, 1], ordered=True)

In [107]: s
Out[107]: 
0    1
1    2
2    3
3    1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [108]: s = s.sort_values()

In [109]: s
Out[109]: 
1    2
2    3
0    1
3    1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [110]: s.min(), s.max()
Out[110]: (np.int64(2), np.int64(1))
```

> **Note**
>
> Note the difference between assigning new categories and reordering the categories: the first renames categories and therefore the individual values in the `Series`, but if the first position was sorted last, the renamed value will still be sorted last. Reordering means that the way values are sorted is different afterwards, but not that individual values in the `Series` are changed.

> **Note**
>
> If the `Categorical` is not ordered, `Series.min()` and `Series.max()` will raise `TypeError`. Numeric operations like `+`, `-`, `*`, `/` and operations based on them (e.g. `Series.median()`, which would need to compute the mean between two values if the length of an array is even) do not work and raise a `TypeError`.

Multi column sorting

A categorical dtyped column will participate in a multi-column sort in a similar manner to other columns. The ordering of the categorical is determined by the `categories` of that column.

```
In [111]: dfs = pd.DataFrame(
   .....:     {
   .....:         "A": pd.Categorical(
   .....:             list("bbeebbaa"),
   .....:             categories=["e", "a", "b"],
   .....:             ordered=True,
   .....:         ),
   .....:         "B": [1, 2, 1, 2, 2, 1, 2, 1],
   .....:     }
   .....: )
   .....: 

In [112]: dfs.sort_values(by=["A", "B"])
Out[112]: 
   A  B
2  e  1
3  e  2
7  a  1
6  a  2
0  b  1
5  b  1
1  b  2
4  b  2
```

Reordering the `categories` changes a future sort.

```
In [113]: dfs["A"] = dfs["A"].cat.reorder_categories(["a", "b", "e"])

In [114]: dfs.sort_values(by=["A", "B"])
Out[114]: 
   A  B
7  a  1
6  a  2
0  b  1
5  b  1
1  b  2
4  b  2
2  e  1
3  e  2
```

Comparisons

Comparing categorical data with other objects is possible in three cases:

- Comparing equality (`==` and `!=`) to a list-like object (list, Series, array, …) of the same length as the categorical data.
- All comparisons (`==`, `!=`, `>`, `>=`, `<`, and `<=`) of categorical data to another categorical Series, when `ordered==True` and the `categories` are the same.
- All comparisons of a categorical data to a scalar.

All other comparisons, especially "non-equality" comparisons of two categoricals with different categories or a categorical with any list-like object, will raise a `TypeError`.

> **Note**
>
> Any "non-equality" comparisons of categorical data with a `Series`, `np.array`, `list` or categorical data with different categories or ordering will raise a `TypeError` because custom categories ordering could be interpreted in two ways: one with taking into account the ordering and one without.

```
In [115]: cat = pd.Series([1, 2, 3]).astype(CategoricalDtype([3, 2, 1], ordered=Tru

In [116]: cat_base = pd.Series([2, 2, 2]).astype(CategoricalDtype([3, 2, 1], ordere

In [117]: cat_base2 = pd.Series([2, 2, 2]).astype(CategoricalDtype(ordered=True))

In [118]: cat
Out[118]: 
0    1
1    2
2    3
dtype: category
Categories (3, int64): [3 < 2 < 1]

In [119]: cat_base
Out[119]: 
0    2
1    2
2    2
dtype: category
Categories (3, int64): [3 < 2 < 1]

In [120]: cat_base2
Out[120]: 
0    2
1    2
2    2
dtype: category
Categories (1, int64): [2]
```

Comparing to a categorical with the same categories and ordering or to a scalar works:

```
In [121]: cat > cat_base
Out[121]: 
0     True
1    False
2    False
dtype: bool

In [122]: cat > 2
Out[122]: 
0     True
1    False
2    False
dtype: bool
```

Equality comparisons work with any list-like object of same length and scalars:

```
In [123]: cat == cat_base
Out[123]: 
0    False
1     True
2    False
dtype: bool

In [124]: cat == np.array([1, 2, 3])
Out[124]: 
0    True
1    True
2    True
dtype: bool

In [125]: cat == 2
Out[125]: 
0    False
1     True
2    False
dtype: bool
```

This doesn't work because the categories are not the same:

```
In [126]: try:
   .....:     cat > cat_base2
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....: 
TypeError: Categoricals can only be compared if 'categories' are the same.
```

If you want to do a "non-equality" comparison of a categorical series with a list-like object which is not categorical data, you need to be explicit and convert the categorical data back to the original values:

```
In [127]: base = np.array([1, 2, 3])

In [128]: try:
   .....:     cat > base
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....: 
TypeError: Cannot compare a Categorical for op __gt__ with type <class 'numpy.ndarra
If you want to compare values, use 'np.asarray(cat) <op> other'.

In [129]: np.asarray(cat) > base
Out[129]: array([False, False, False])
```

When you compare two unordered categoricals with the same categories, the order is not considered:

```
In [130]: c1 = pd.Categorical(["a", "b"], categories=["a", "b"], ordered=False)

In [131]: c2 = pd.Categorical(["a", "b"], categories=["b", "a"], ordered=False)

In [132]: c1 == c2
Out[132]: array([ True,  True])
```

Operations

Apart from `Series.min()`, `Series.max()` and `Series.mode()`, the following operations are possible with categorical data:

`Series` methods like `Series.value_counts()` will use all categories, even if some categories are not present in the data:

```
In [133]: s = pd.Series(pd.Categorical(["a", "b", "c", "c"], categories=["c", "a", "

In [134]: s.value_counts()
Out[134]: 
c    2
a    1
b    1
d    0
Name: count, dtype: int64
```

`DataFrame` methods like `DataFrame.sum()` also show "unused" categories when `observed=False`.

```
In [135]: columns = pd.Categorical(
   .....:     ["One", "One", "Two"], categories=["One", "Two", "Three"], ordered=Tr
   .....: )
   .....: 

In [136]: df = pd.DataFrame(
   .....:     data=[[1, 2, 3], [4, 5, 6]],
   .....:     columns=pd.MultiIndex.from_arrays([["A", "B", "B"], columns]),
   .....: ).T
   .....: 

In [137]: df.groupby(level=1, observed=False).sum()
Out[137]: 
       0  1
One    3  9
Two    3  6
Three  0  0
```

Groupby will also show "unused" categories when `observed=False`:

```
In [138]: cats = pd.Categorical(
   .....:     ["a", "b", "b", "b", "c", "c", "c"], categories=["a", "b", "c", "d"]
   .....: )
   .....: 

In [139]: df = pd.DataFrame({"cats": cats, "values": [1, 2, 2, 2, 3, 4, 5]})

In [140]: df.groupby("cats", observed=False).mean()
Out[140]: 
      values
cats        
a        1.0
b        2.0
c        4.0
d        NaN

In [141]: cats2 = pd.Categorical(["a", "a", "b", "b"], categories=["a", "b", "c"])

In [142]: df2 = pd.DataFrame(
   .....:     {
   .....:         "cats": cats2,
   .....:         "B": ["c", "d", "c", "d"],
   .....:         "values": [1, 2, 3, 4],
   .....:     }
   .....: )
   .....: 

In [143]: df2.groupby(["cats", "B"], observed=False).mean()
Out[143]: 
        values
cats B        
a    c     1.0
     d     2.0
b    c     3.0
     d     4.0
c    c     NaN
     d     NaN
```

Pivot tables:

```
In [144]: raw_cat = pd.Categorical(["a", "a", "b", "b"], categories=["a", "b", "c"]

In [145]: df = pd.DataFrame({"A": raw_cat, "B": ["c", "d", "c", "d"], "values": [1,

In [146]: pd.pivot_table(df, values="values", index=["A", "B"], observed=False)
Out[146]: 
     values
A B        
a c     1.0
  d     2.0
b c     3.0
  d     4.0
```

Data munging

The optimized pandas data access methods `.loc`, `.iloc`, `.at`, and `.iat`, work as normal. The only difference is the return type (for getting) and that only values already in `categories` can be assigned.

Getting

If the slicing operation returns either a `DataFrame` or a column of type `Series`, the `category` dtype is preserved.

```
In [147]: idx = pd.Index(["h", "i", "j", "k", "l", "m", "n"])

In [148]: cats = pd.Series(["a", "b", "b", "b", "c", "c", "c"], dtype="category", i

In [149]: values = [1, 2, 2, 2, 3, 4, 5]

In [150]: df = pd.DataFrame({"cats": cats, "values": values}, index=idx)

In [151]: df.iloc[2:4, :]
Out[151]: 
  cats  values
j    b       2
k    b       2

In [152]: df.iloc[2:4, :].dtypes
Out[152]: 
cats      category
values       int64
dtype: object

In [153]: df.loc["h":"j", "cats"]
Out[153]: 
h    a
i    b
j    b
Name: cats, dtype: category
Categories (3, str): ['a', 'b', 'c']

In [154]: df[df["cats"] == "b"]
Out[154]: 
  cats  values
i    b       2
j    b       2
k    b       2
```

An example where the category type is not preserved is if you take one single row: the resulting `Series` is of dtype `object`:

```
# get the complete "h" row as a Series
In [155]: df.loc["h", :]
Out[155]: 
cats      a
values    1
Name: h, dtype: object
```

Returning a single item from categorical data will also return the value, not a categorical of length "1".

```
In [156]: df.iat[0, 0]
Out[156]: 'a'

In [157]: df["cats"] = df["cats"].cat.rename_categories(["x", "y", "z"])

In [158]: df.at["h", "cats"]  # returns a string
Out[158]: 'x'
```

> **Note**
>
> This is in contrast to R's `factor` function, where `factor(c(1,2,3))[1]` returns a single value `factor`.

To get a single value `Series` of type `category`, you pass in a list with a single value:

```
In [159]: df.loc[["h"], "cats"]
Out[159]: 
h    x
Name: cats, dtype: category
Categories (3, str): ['x', 'y', 'z']
```

String and datetime accessors

The accessors `.dt` and `.str` will work if the `s.cat.categories` are of an appropriate type:

```
In [160]: str_s = pd.Series(list("aabb"))

In [161]: str_cat = str_s.astype("category")

In [162]: str_cat
Out[162]: 
0    a
1    a
2    b
3    b
dtype: category
Categories (2, str): ['a', 'b']

In [163]: str_cat.str.contains("a")
Out[163]: 
0     True
1     True
2    False
3    False
dtype: bool

In [164]: date_s = pd.Series(pd.date_range("1/1/2015", periods=5))

In [165]: date_cat = date_s.astype("category")

In [166]: date_cat
Out[166]: 
0   2015-01-01
1   2015-01-02
2   2015-01-03
3   2015-01-04
4   2015-01-05
dtype: category
Categories (5, datetime64[us]): [2015-01-01, 2015-01-02, 2015-01-03, 2015-01-04, 20

In [167]: date_cat.dt.day
Out[167]: 
0    1
1    2
2    3
3    4
4    5
dtype: int32
```

> **Note**
>
> The returned `Series` (or `DataFrame`) is of the same type as if you used the `.str.<method>` / `.dt.<method>` on a `Series` of that type (and not of type `category`!).

That means, that the returned values from methods and properties on the accessors of a `Series` and the returned values from methods and properties on the accessors of this `Series` transformed to one of type `category` will be equal:

```
In [168]: ret_s = str_s.str.contains("a")

In [169]: ret_cat = str_cat.str.contains("a")

In [170]: ret_s.dtype == ret_cat.dtype
Out[170]: True

In [171]: ret_s == ret_cat
Out[171]: 
0    True
1    True
2    True
3    True
dtype: bool
```

> **Note**
>
> The work is done on the `categories` and then a new `Series` is constructed. This has some performance implication if you have a `Series` of type string, where lots of elements are repeated (i.e. the number of unique elements in the `Series` is a lot smaller than the length of the `Series`). In this case it can be faster to convert the original `Series` to one of type `category` and use `.str.<method>` or `.dt.<property>` on that.

Setting

Setting values in a categorical column (or `Series`) works as long as the value is included in the `categories`:

```
In [172]: idx = pd.Index(["h", "i", "j", "k", "l", "m", "n"])

In [173]: cats = pd.Categorical(["a", "a", "a", "a", "a", "a", "a"], categories=["a"

In [174]: values = [1, 1, 1, 1, 1, 1, 1]

In [175]: df = pd.DataFrame({"cats": cats, "values": values}, index=idx)

In [176]: df.iloc[2:4, :] = [["b", 2], ["b", 2]]

In [177]: df
Out[177]: 
  cats  values
h    a       1
i    a       1
j    b       2
k    b       2
l    a       1
m    a       1
n    a       1

In [178]: try:
   .....:     df.iloc[2:4, :] = [["c", 3], ["c", 3]]
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....: 
TypeError: Cannot setitem on a Categorical with a new category, set the categories
```

Setting values by assigning categorical data will also check that the `categories` match:

```
In [179]: df.loc["j":"k", "cats"] = pd.Categorical(["a", "a"], categories=["a", "b"

In [180]: df
Out[180]: 
  cats  values
h    a       1
i    a       1
j    a       2
k    a       2
l    a       1
m    a       1
n    a       1

In [181]: try:
   .....:     df.loc["j":"k", "cats"] = pd.Categorical(["b", "b"], categories=["a",
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....: 
TypeError: Cannot set a Categorical with another, without identical categories
```

Assigning a `Categorical` to parts of a column of other types will use the values:

```
In [182]: df = pd.DataFrame({"a": [1, 1, 1, 1, 1], "b": ["a", "a", "a", "a", "a"]})

In [183]: df.loc[1:2, "a"] = pd.Categorical([2, 2], categories=[2, 3])

In [184]: df.loc[2:3, "b"] = pd.Categorical(["b", "b"], categories=["a", "b"])

In [185]: df
Out[185]: 
   a  b
0  1  a
1  2  a
2  2  b
3  1  b
4  1  a

In [186]: df.dtypes
Out[186]: 
a    int64
b      str
dtype: object
```

Merging / concatenation

By default, combining `Series` or `DataFrames` which contain the same categories results in `category` dtype, otherwise results will depend on the dtype of the underlying categories. Merges that result in non-categorical dtypes will likely have higher memory usage. Use `.astype` or `union_categoricals` to ensure `category` results.

```
In [187]: from pandas.api.types import union_categoricals

# same categories
In [188]: s1 = pd.Series(["a", "b"], dtype="category")

In [189]: s2 = pd.Series(["a", "b", "a"], dtype="category")

In [190]: pd.concat([s1, s2])
Out[190]: 
0    a
1    b
0    a
1    b
2    a
dtype: category
Categories (2, str): ['a', 'b']

# different categories
In [191]: s3 = pd.Series(["b", "c"], dtype="category")

In [192]: pd.concat([s1, s3])
Out[192]: 
0    a
1    b
0    b
1    c
dtype: str

# Output dtype is inferred based on categories values
In [193]: int_cats = pd.Series([1, 2], dtype="category")

In [194]: float_cats = pd.Series([3.0, 4.0], dtype="category")

In [195]: pd.concat([int_cats, float_cats])
Out[195]: 
0    1.0
1    2.0
0    3.0
1    4.0
dtype: float64

In [196]: pd.concat([s1, s3]).astype("category")
Out[196]: 
0    a
1    b
0    b
1    c
dtype: category
Categories (3, str): ['a', 'b', 'c']

In [197]: union_categoricals([s1.array, s3.array])
Out[197]: 
['a', 'b', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
```

The following table summarizes the results of merging `Categoricals`:

arg1	arg2	identical	result
category	category	True	category
category (object)	category (object)	False	object (dtype is inferred)
category (int)	category (float)	False	float (dtype is inferred)

Unioning

If you want to combine categoricals that do not necessarily have the same categories, the `union_categoricals()` function will combine a list-like of categoricals. The new categories will be the union of the categories being combined.

```
In [198]: from pandas.api.types import union_categoricals

In [199]: a = pd.Categorical(["b", "c"])

In [200]: b = pd.Categorical(["a", "b"])

In [201]: union_categoricals([a, b])
Out[201]: 
['b', 'c', 'a', 'b']
Categories (3, str): ['b', 'c', 'a']
```

By default, the resulting categories will be ordered as they appear in the data. If you want the categories to be lexsorted, use `sort_categories=True` argument.

```
In [202]: union_categoricals([a, b], sort_categories=True)
Out[202]: 
['b', 'c', 'a', 'b']
Categories (3, str): ['a', 'b', 'c']
```

`union_categoricals` also works with the "easy" case of combining two categoricals of the same categories and order information (e.g. what you could also `append` for).

```
In [203]: a = pd.Categorical(["a", "b"], ordered=True)

In [204]: b = pd.Categorical(["a", "b", "a"], ordered=True)

In [205]: union_categoricals([a, b])
Out[205]: 
['a', 'b', 'a', 'b', 'a']
Categories (2, str): ['a' < 'b']
```

The below raises `TypeError` because the categories are ordered and not identical.

```
In [206]: a = pd.Categorical(["a", "b"], ordered=True)

In [207]: b = pd.Categorical(["a", "b", "c"], ordered=True)

In [208]: union_categoricals([a, b])
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[208], line 1
----> 1 union_categoricals([a, b])

File ~/work/pandas/pandas/pandas/core/dtypes/concat.py:335, in union_categoricals(t
    333     if all(c.ordered for c in to_union):
```

```
    334         msg = "to union ordered Categoricals, all categories must be the sa
--> 335         raise TypeError(msg)
    336     raise TypeError("Categorical.ordered must be the same")
    338 if ignore_order:

TypeError: to union ordered Categoricals, all categories must be the same
```

Ordered categoricals with different categories or orderings can be combined by using the `ignore_ordered=True` argument.

```
In [209]: a = pd.Categorical(["a", "b", "c"], ordered=True)

In [210]: b = pd.Categorical(["c", "b", "a"], ordered=True)

In [211]: union_categoricals([a, b], ignore_order=True)
Out[211]:
['a', 'b', 'c', 'c', 'b', 'a']
Categories (3, str): ['a', 'b', 'c']
```

`union_categoricals()` also works with a `CategoricalIndex`, or `Series` containing categorical data, but note that the resulting array will always be a plain `Categorical`:

```
In [212]: a = pd.Series(["b", "c"], dtype="category")

In [213]: b = pd.Series(["a", "b"], dtype="category")

In [214]: union_categoricals([a, b])
Out[214]:
['b', 'c', 'a', 'b']
Categories (3, str): ['b', 'c', 'a']
```

Note

`union_categoricals` may recode the integer codes for categories when combining categoricals. This is likely what you want, but if you are relying on the exact numbering of the categories, be aware.

```
In [215]: c1 = pd.Categorical(["b", "c"])

In [216]: c2 = pd.Categorical(["a", "b"])

In [217]: c1
Out[217]:
['b', 'c']
Categories (2, str): ['b', 'c']

# "b" is coded to 0
In [218]: c1.codes
Out[218]: array([0, 1], dtype=int8)

In [219]: c2
Out[219]:
['a', 'b']
Categories (2, str): ['a', 'b']

# "b" is coded to 1
In [220]: c2.codes
Out[220]: array([0, 1], dtype=int8)

In [221]: c = union_categoricals([c1, c2])

In [222]: c
Out[222]:
['b', 'c', 'a', 'b']
Categories (3, str): ['b', 'c', 'a']

# "b" is coded to 0 throughout, same as c1, different from c2
In [223]: c.codes
Out[223]: array([0, 1, 2, 0], dtype=int8)
```

Getting data in/out

You can write data that contains `category` dtypes to a `HDFStore`. See here for an example and caveats.

It is also possible to write data to and reading data from *Stata* format files. See here for an example and caveats.

Writing to a CSV file will convert the data, effectively removing any information about the categorical (categories and ordering). So if you read back the CSV file you have to convert the relevant columns back to `category` and assign the right categories and categories ordering.

```
In [224]: import io

In [225]: s = pd.Series(pd.Categorical(["a", "b", "b", "a", "a", "d"]))

# rename the categories
In [226]: s = s.cat.rename_categories(["very good", "good", "bad"])

# reorder the categories and add missing categories
In [227]: s = s.cat.set_categories(["very bad", "bad", "medium", "good", "very good"

In [228]: df = pd.DataFrame({"cats": s, "vals": [1, 2, 3, 4, 5, 6]})

In [229]: csv = io.StringIO()

In [230]: df.to_csv(csv)

In [231]: df2 = pd.read_csv(io.StringIO(csv.getvalue()))

In [232]: df2.dtypes
Out[232]:
Unnamed: 0    int64
cats            str
vals          int64
dtype: object

In [233]: df2["cats"]
Out[233]:
0    very good
1         good
2         good
3    very good
4    very good
5          bad
Name: cats, dtype: str

# Redo the category
In [234]: df2["cats"] = df2["cats"].astype("category")

In [235]: df2["cats"] = df2["cats"].cat.set_categories(
   .....:     ["very bad", "bad", "medium", "good", "very good"]
   .....: )
   .....:

In [236]: df2.dtypes
Out[236]:
Unnamed: 0       int64
cats          category
vals             int64
dtype: object

In [237]: df2["cats"]
Out[237]:
0    very good
1         good
2         good
3    very good
4    very good
5          bad
Name: cats, dtype: category
Categories (5, str): ['very bad', 'bad', 'medium', 'good', 'very good']
```

The same holds for writing to a SQL database with `to_sql`.

Missing data

pandas primarily uses the value `np.nan` to represent missing data. It is by default not included in computations. See the Missing Data section.

Missing values should **not** be included in the Categorical's `categories`, only in the `values`. Instead, it is understood that NaN is different, and is always a possibility. When working with the Categorical's `codes`, missing values will always have a code of `-1`.

```
In [238]: s = pd.Series(["a", "b", np.nan, "a"], dtype="category")

# only two categories
In [239]: s
Out[239]:
0      a
1      b
2    NaN
3      a
dtype: category
Categories (2, str): ['a', 'b']

In [240]: s.cat.codes
Out[240]:
0    0
1    1
2   -1
3    0
dtype: int8
```

Methods for working with missing data, e.g. **`isna()`**, **`fillna()`**, **`dropna()`**, all work normally:

```
In [241]: s = pd.Series(["a", "b", np.nan], dtype="category")

In [242]: s
Out[242]:
0      a
1      b
2    NaN
dtype: category
Categories (2, str): ['a', 'b']

In [243]: pd.isna(s)
Out[243]:
0    False
1    False
2     True
dtype: bool

In [244]: s.fillna("a")
Out[244]:
0    a
1    b
2    a
dtype: category
Categories (2, str): ['a', 'b']
```

Differences to R's `factor`

The following differences to R's factor functions can be observed:

- R's `levels` are named `categories`.
- R's `levels` are always of type string, while `categories` in pandas can be of any dtype.
- It's not possible to specify labels at creation time. Use `s.cat.rename_categories(new_labels)` afterwards.
- In contrast to R's `factor` function, using categorical data as the sole input to create a new categorical series will *not* remove unused categories but create a new categorical series which is equal to the passed in one!
- R allows for missing values to be included in its `levels` (pandas' `categories`). pandas does not allow `NaN` categories, but missing values can still be in the `values`.

Gotchas

Memory usage

The memory usage of a `Categorical` is proportional to the number of categories plus the length of the data. In contrast, an `object` dtype is a constant times the length of the data.

```
In [245]: s = pd.Series(["foo", "bar"] * 1000)

# object dtype
In [246]: s.nbytes
Out[246]: 22000

# category dtype
In [247]: s.astype("category").nbytes
Out[247]: 2023
```

Note

If the number of categories approaches the length of the data, the `Categorical` will use nearly the same or more memory than an equivalent `object` dtype representation.

```
In [248]: s = pd.Series(["foo%04d" % i for i in range(2000)])

# object dtype
In [249]: s.nbytes
Out[249]: 30000

# category dtype
In [250]: s.astype("category").nbytes
Out[250]: 34250
```

`Categorical` is not a `numpy` array

Currently, categorical data and the underlying `Categorical` is implemented as a Python object and not as a low-level NumPy array dtype. This leads to some problems.

NumPy itself doesn't know about the new `dtype`:

```
In [251]: try:
   .....:     np.dtype("category")
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....:
TypeError: data type 'category' not understood

In [252]: dtype = pd.Categorical(["a"]).dtype

In [253]: try:
   .....:     np.dtype(dtype)
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....:
TypeError: Cannot interpret 'CategoricalDtype(categories=['a'], ordered=False, cate
```

Dtype comparisons work:

```
In [254]: dtype == np.str_
Out[254]: False

In [255]: np.str_ == dtype
Out[255]: False
```

To check if a Series contains Categorical data, use `hasattr(s, 'cat')`:

```
In [256]: hasattr(pd.Series(["a"], dtype="category"), "cat")
Out[256]: True

In [257]: hasattr(pd.Series(["a"]), "cat")
Out[257]: False
```

Using NumPy functions on a `Series` of type `category` should not work as `Categoricals` are not numeric data (even in the case that `.categories` is numeric).

```
In [258]: s = pd.Series(pd.Categorical([1, 2, 3, 4]))

In [259]: try:
   .....:     np.sum(s)
   .....: except TypeError as e:
   .....:     print("TypeError:", str(e))
   .....:
TypeError: 'Categorical' with dtype category does not support operation 'sum'
```

Note

If such a function works, please file a bug at pandas-dev/pandas!

dtype in apply

pandas currently does not preserve the dtype in apply functions: If you apply along rows you get a `Series` of `object` `dtype` (same as getting a row -> getting one element will return a basic type) and applying along columns will also convert to object. `NaN` values are unaffected. You can use `fillna` to handle missing values before applying a function.

```
In [260]: df = pd.DataFrame(
   .....:     {
   .....:         "a": [1, 2, 3, 4],
   .....:         "b": ["a", "b", "c", "d"],
   .....:         "cats": pd.Categorical([1, 2, 3, 2]),
   .....:     }
   .....: )
   .....:

In [261]: df.apply(lambda row: type(row["cats"]), axis=1)
Out[261]:
0    <class 'int'>
1    <class 'int'>
2    <class 'int'>
3    <class 'int'>
dtype: object

In [262]: df.apply(lambda col: col.dtype, axis=0)
Out[262]:
a          int64
b            str
cats    category
dtype: object
```

Categorical index

`CategoricalIndex` is a type of index that is useful for supporting indexing with duplicates. This is a container around a `Categorical` and allows efficient indexing and storage of an index with a large number of duplicated elements. See the advanced indexing docs for a more detailed explanation.

Setting the index will create a `CategoricalIndex`:

```
In [263]: cats = pd.Categorical([1, 2, 3, 4], categories=[4, 2, 3, 1])

In [264]: strings = ["a", "b", "c", "d"]

In [265]: values = [4, 2, 3, 1]

In [266]: df = pd.DataFrame({"strings": strings, "values": values}, index=cats)

In [267]: df.index
Out[267]: CategoricalIndex([1, 2, 3, 4], categories=[4, 2, 3, 1], ordered=False, dt

# This now sorts by the categories order
In [268]: df.sort_index()
Out[268]:
  strings  values
4       d       1
2       b       2
3       c       3
1       a       4
```

‹ Previous
Duplicate Labels

Next ›
Nullable integer data type

© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud.

Created using Sphinx 9.0.4.

Built with the PyData Sphinx Theme 0.16.1.